



P2P Systems and Blockchains

-Lab-

Damiano Di Francesco Maesa
17/3/2026
damiano.difrancesco@unipi.it



UNIVERSITÀ
DI PISA

Lecture 4 - Bitcoin lab

bitcoinj - transactions

```
1 public static void readRawTxAndDecode(String rawHexTx) {
2     //String rawHexTx = "020000000180b98c54dbab5106d5a1449f4e5fdb9146deca1d48e93d666c5d9290b7c37a3f010000006b
483045022100f0e32ceb205a5056694611afcffe4c1f0e63e9c57382607045ff2c3d9b5b7b3f0220111f0323e56d7462a929983316656
9f1a68e1f5090b49bea64f541c494109c6c012102d0648f06a31d47112f1ff7848c85ce54b772c513bc3337c98f081c19d3dca260ffff
ffff02006d7c4d000000001976a9147d463a046e3175142464740bad692fa0762a93e88accad5e5f1b50000001976a914c98fc6bd9c2
fd88533f28e6797cfa2a0a0e18ecf88ac00000000";
3
4     byte[] ba = hexStringToByteArray(rawHexTx);
5     ByteBuffer bf = ByteBuffer.wrap(ba);
6     Transaction tx = Transaction.read(bf);
7
8     //Transaction tx = Transaction(rawHexTx);
9
10    for (TransactionInput in: tx.getInputs()) {
11        byte[] inScript = in.getScriptBytes();
12        System.out.println("INPUT script hex " + bytesToHex(inScript));
13        System.out.println("INPUT script OPcodes " + bytesToOpcode(inScript));
14    }
15
16    for (TransactionOutput out: tx.getOutputs()) {
17        byte[] outScript = out.getScriptBytes();
18        System.out.println("OUTPUT script hex " + bytesToHex(outScript));
19        System.out.println("OUTPUT script OPcodes " + bytesToOpcode(outScript));
20        System.out.println("OUTPUT script Type " + ScriptParser.typeFromOut(outScript));
21        String outAddr = ScriptParser.addrFromOut(outScript);
22        if (outAddr == null) {
23            outAddr = "#UNKNOWN#";
24        }
25        System.out.println("OUTPUT script address " + outAddr);
26    }
27 }
28 }
```

bitcoinj - transactions

```
1 public class ScriptTypeCustom {
2     //type of script used 1=P2PK 2=P2PKH 3=P2SH 4=P2MS 5=other
3     public static final int UNKNOWN = 0;
4     public static final int P2PK = 1;
5     public static final int P2PKH = 2;
6     public static final int P2SH = 3;
7     public static final int RETURN = 4;
8     public static final int EMPTY = 5;
9     public static final int P2WPKH = 6;
10    public static final int P2WSH = 7;
11
12    public static final int SUPPORTEDSCRIPTTYPES = 8;
13
14    //multisig example: txhash da738e29f64e90ae46dcc3e6b4154041d6324abbe7919e722d486a4a3148b7dc
15
16    public static String typeName(int code) {
17        switch (code) {
18            case UNKNOWN:
19                return "UNKNOWN";
20            case P2PK:
21                return "P2PK";
22            case P2PKH:
23                return "P2PKH";
24            case P2SH:
25                return "P2SH";
26            case RETURN:
27                return "PROBABLY UNSPENDABLE";
28            case EMPTY:
29                return "ANYONE CAN SPEND";
30            case P2WPKH:
31                return "P2WPKH";
32            case P2WSH:
33                return "P2WSH";
34            default:
35                return "ERROR - UNRECOGNIZED SCRIPT CODE";
36        }
37    }
38 }
```

bitcoinj - transactions

```
1 public static int typeFromOut(byte[] script) {
2     if ((script == null) || (script.length < 1))
3         return ScriptTypeCustom.EMPTY;
4     //no empty script
5     if (isOpCode(script[0], OP_RETURN))
6         return ScriptTypeCustom.RETURN;
7     else if (isOpCode(script[0], OP_DUP) && (script.length >= 23)) {
8         //P2PBKH
9         //System.out.println("P2PBKH");
10        if (isOpCode(script[1], OP_HASH160) && isOpCode(script[2], OP_PUSHDATA20)) {
11            return ScriptTypeCustom.P2PKH;
12        } else
13            return ScriptTypeCustom.UNKNOWN;
14    } else if (isOpCode(script[0], OP_PUSHDATA65) && (script.length >= 66)) {
15        return ScriptTypeCustom.P2PK;
16    } else if ((script.length == 66) && ((isOpCode(script[script.length - 1], OP_CHECKSIG) || (isOpCode(
17        script[script.length - 1], OP_CHECKSIGVERIFY)))) {
18        //old broken P2PBK
19        return ScriptTypeCustom.P2PK;
20    } else if (isOpCode(script[0], OP_HASH160)) {
21        if ((script.length >= 23) && isOpCode(script[1], OP_PUSHDATA20) && (isOpCode(script[script.length -
22        1], OP_EQUAL) || isOpCode(script[script.length - 1], OP_EQUALVERIFY))) {
23            //P2SH 160
24            return ScriptTypeCustom.P2SH;
25        } else return ScriptTypeCustom.UNKNOWN;
26    } else if (isOpCode(script[0], OP_0)) {
27        //support for native segwit
28        if (isOpCode(script[1], OP_PUSHDATA20) && (script.length == 22)) {
29            //P2WPKH
30            return ScriptTypeCustom.P2WPKH;
31        } else if (isOpCode(script[1], OP_PUSHDATA32) && (script.length == 34)) {
32            //P2WSH
33            return ScriptTypeCustom.P2WSH;
34        } else {
35            return ScriptTypeCustom.UNKNOWN;
36        }
37    } else
38        return ScriptTypeCustom.UNKNOWN;
39 }
```

```
1 public class ScriptParser {
2     private static final int OP_DUP = 118;
3     private static final int OP_HASH160 = 169;
4     private static final int OP_EQUALVERIFY = 136;
5     private static final int OP_CHECKSIG = 172;
6     private static final int OP_CHECKSIGVERIFY = 173;
7     private static final int OP_EQUAL = 135;
8     private static final int OP_RETURN = 106;
9     //between 1 and 75 it is a relevant op_push_data0 i.e. number of bytes to be pushed to the stack
10    private static final int OP_PUSHDATA20 = 20;
11    private static final int OP_PUSHDATA32 = 32;
12    private static final int OP_PUSHDATA33 = 33;
13    private static final int OP_PUSHDATA65 = 65;
14    private static final int OP_0 = 0;
15
16    private static boolean isOpCode(byte b, int opcode) {
17        return Utilities.readUnsignedByte(b) == opcode;
18    }
19 }
```

bitcoinj - transactions

```
1 public static String addrFromOut(byte[] script) {
2     if ((script == null) || (script.length < 1)) return null;
3     //no empty script
4     if (isOpCode(script[0], OP_RETURN))
5         return null;
6     else if (isOpCode(script[0], OP_DUP) && (script.length >= 23)) {
7         //it is P2PBKH
8         //System.out.println("P2PBKH");
9         if (isOpCode(script[1], OP_HASH160) && isOpCode(script[2], OP_PUSHDATA20)) {
10             //System.out.println("P2PBKH--in");
11             byte[] res = new byte[20];
12             System.arraycopy(script, 3, res, 0, 20);
13             return getAddressFromPubHash(res);
14         } else
15             return null;
16     } else if (isOpCode(script[0], OP_PUSHDATA65) && (script.length >= 66)) {
17         //it is P2PBK
18         byte[] res = new byte[65];
19         System.arraycopy(script, 1, res, 0, 65);
20         return getAddressFromPubKey(res);
21     } else if ((script.length == 66) && ((isOpCode(script[script.length - 1], OP_CHECKSIG) || (isOpCode(
22 script[script.length - 1], OP_CHECKSIGVERIFY)))) {
23         //it is old broken version of P2PBK without initial length byte
24         byte[] res = new byte[65];
25         System.arraycopy(script, 0, res, 0, 65);
26         return getAddressFromPubKey(res);
27     } else if (isOpCode(script[0], OP_HASH160)) {
28         if ((script.length >= 23) && isOpCode(script[1], OP_PUSHDATA20) && (isOpCode(script[script.length
29 1], OP_EQUAL) || isOpCode(script[script.length - 1], OP_EQUALVERIFY))) {
30             //it is P2SH 160
31             byte[] res = new byte[20];
32             System.arraycopy(script, 2, res, 0, 20);
33             return getAddressFromScriptHash(res);
34         } else return null;
35     } else if (isOpCode(script[0], OP_0)) {
36         //support for native segwit
37         if (isOpCode(script[1], OP_PUSHDATA20) && (script.length == 22)) {
38             //P2WPKH
39             byte[] res = new byte[20];
40             System.arraycopy(script, 2, res, 0, 20);
41             return SegwitAddress.fromHash(MainNetParams.get(), res).toBech32();
42         } else if (isOpCode(script[1], OP_PUSHDATA32) && (script.length == 34)) {
43             //P2WSH
44             byte[] res = new byte[32];
45             System.arraycopy(script, 2, res, 0, 32);
46             return SegwitAddress.fromHash(MainNetParams.get(), res).toBech32();
47         } else {
48             return null;
49         }
50     } else
51         return null;
52 }
```

```
1 //PRE:b is long 20
2 public static String getAddressFromPubHash(byte[] b) {
3     //add version "00"
4     byte[] version = {
5         0
6     };
7     //base58check encoding
8     return Base58.encodeChecked(version[0], b);
9 }
10 //PRE:b is long 65
11 public static String getAddressFromPubKey(byte[] b) {
12     //get hash160 from pubkey
13     //perform sha256
14     //perform ripemd160
15     //encode hash160
16     return getAddressFromPubHash(sha256hash160(b));
17 }
18 //PRE:b is long 20
19 public static String getAddressFromScriptHash(byte[] b) {
20     //add version "00"
21     byte[] version = {
22         5
23     };
24     //base58check encoding
25     return Base58.encodeChecked(version[0], b);
26 }
```

bitcoinj - transactions

Extend to P2TR

<https://learnmeabitcoin.com/technical/script/p2tr/>

bitcoin core - blk.dat

<https://learnmeabitcoin.com/technical/block/blkdat/>

bitcoinj - blk.dat

```
1 public class BCParser {
2
3     // Location of block files
4     String chaindataFolder;
5     int DEBUGtotalTxs;
6     int DEBUGcoinbaseCounter;
7     int DEBUGwitnessTxs;
8     int DEBUGnullAddresses;
9     int[] DEBUGscriptTypes;
10
11     public BCParser(String f) {
12         chaindataFolder = f;
13         DEBUGtotalTxs = 0;
14         DEBUGcoinbaseCounter = 0;
15         DEBUGwitnessTxs = 0;
16         DEBUGnullAddresses = 0;
17         DEBUGscriptTypes = new int[ScriptTypeCustom.SUPPORTEDSCRIPTTYPES];
18         for (int i = 0; i < DEBUGscriptTypes.length; i++)
19             DEBUGscriptTypes[i] = 0;
20     }
21
22     // The method returns a list of files in a directory according to a certain
23     // pattern (block files have name blkNNNNN.dat)
24     public static List < File > buildList(String PREFIX) {
25         List < File > list = new LinkedList < File > ();
26         for (int i = 0; true; i++) {
27             File file = new File(PREFIX + String.format(Locale.US, "blk%05d.dat", i));
28             if (!file.exists())
29                 break;
30             list.add(file);
31         }
32         return list;
33     }
34 }
```

bitcoinj- blk.dat

```
1  /**
2   *
3   * @param out file where to save the list of transactions from all blk files in the folder
4   * @param n max number of blocks (NOT ORDERED!!) to parse
5   * @throws IOException
6   */
7  public void parseNoUtxo(File out, int n) throws IOException {
8      NetworkParameters np = MainNetParams.get();
9      //Creates a BlockFileLoader object by passing a list of .dat files.
10     BlockFileLoader loader = new BlockFileLoader(np, buildList(chaindataFolder));
11     //BlockFileLoader loaderNew = new BlockFileLoader(BitcoinNetwork.MAINNET, Utilities.buildList(chaindataFolder));
12     BufferedWriter bw = new BufferedWriter(new FileWriter(out));
13
14     int blockCounter = 0;
15     //NOTE: blocks are not ordered, so this is NOT the same as block height!!
16     for (Block block: loader) {
17         if (blockCounter >= n)
18             break;
19         if (blockCounter % 20000 == 0) {
20             System.out.println("Analysed " + blockCounter + " NOT ORDERED blocks.");
21             System.out.println(blockCounter + " - " + block.getHashAsString());
22         }
23         parseBlockExact(block, bw);
24         blockCounter++;
25     } // End of iteration over blocks
26     bw.close();
27     System.out.println("TotalTxs " + DEBUGtotalTxs + ", of which coinbases are " + DEBUGcoinbaseCounter + "
28     and " + DEBUGwitnessTxs + " are witness transactions.");
29     System.out.println("Scripts found :");
30     int ttemp = 0;
31     for (int i = 0; i < DEBUGscriptTypes.length; i++) {
32         System.out.println(DEBUGscriptTypes[i] + " " + ScriptTypeCustom.typeName(i));
33         ttemp += DEBUGscriptTypes[i];
34     }
35     System.out.println("Total : " + ttemp + " (" + DEBUGnullAddresses + " null addresses).");
36 }
```

bitcoinj- blk.dat

```
1  /**
2   * Outputs script info for the given block as:
3   * one tx per line
4   * generalInfo ': ' InputsInfo (empty if coinbase) ': ' OutputsInfo
5   * generalInfo := timeStamp, ' blockHash', ' txHash', ' isCoinbase', ' txSizeEstimate
6   *
7   * @param block
8   * @param bw
9   * @throws IOException
10  */
11 public void parseBlockExact(Block block, BufferedWriter bw) throws IOException {
12     boolean isCoinbase;
13     boolean first;
14     StringBuilder line;
15     for (Transaction tx: block.getTransactions()) {
16         DEBUGtotalTxns++;
17         //write tx general infos:
18         //timestamp, blockhash, txHash, isCoinbase, estimatedSize, hasWitness:
19         line = new StringBuilder();
20         line.append(block.time());
21         line.append(",");
22         line.append(block.getHashAsString());
23         line.append(",");
24         line.append(tx.getTxId().toString());
25         line.append(",");
26         if (tx.isCoinbase()) {
27             isCoinbase = true;
28             line.append("1");
29         } else {
30             isCoinbase = false;
31             line.append("0");
32         }
33         line.append(",");
34         line.append(tx.getVsize());
35         line.append(",");
36         if (tx.hasWitnesses()) {
37             DEBUGwitnessTxns++;
38             line.append("1");
39         } else {
40             line.append("0");
41         }
42         line.append(":");
43         if (isCoinbase) {
44             DEBUGcoinbaseCounter++;
45         } else {
46             //not coinbase so there is at least one input
47             //save inputs in the format
48             //[[prevTx_Id, prevTxPos[:]]]*:
49             first = true;
50             for (TransactionInput ii: tx.getInputs()) {
51                 if (first)
52                     first = false;
53                 else
54                     line.append(",");
55                 //line.append(Utilities.byteArrayToHexString(ii.getScriptBytes()));
56                 line.append(ii.getOutput().hash().toString());
57                 line.append(", " + ii.getOutput().index());
58             }
59     }
```

```
60     line.append(";");
61     //save outputs in the format
62     //[[addr, amount, scriptType][:addr, amount, scriptType]*
63     //there is always at least one output
64     first = true;
65     for (TransactionOutput oo: tx.getOutputs()) {
66         if (first)
67             first = false;
68         else
69             line.append(",");
70         byte[] outScript = oo.getScriptBytes();
71         String outAddr = ScriptParser.addrFromOut(outScript);
72         int outType = ScriptParser.typeFromOut(outScript);
73         DEBUGscriptTypes[outType]++;
74         if (outAddr == null) {
75             //writes '#num' as address if not decodable
76             outAddr = "#" + DEBUGnullAddresses;
77             DEBUGnullAddresses++;
78         }
79         line.append(outAddr);
80         line.append(", " + oo.getValue().getValue());
81         line.append(", " + outType);
82     }
83     bw.write(line.toString());
84     bw.newLine();
85 }
86 }
```

Questions

